Backend API — Project Documentation

Version 1.0 · May 2026

Version: 1.0

Date: May 2026

Author: FreelanceHub Development Team

Repository: BackendFreelance

Table of Contents

1. Project Overview
2. Objectives
3. Features and Functionalities
4. System Architecture
5. Technologies Used
6. Installation and Setup
7. Usage Guide
8. API Endpoints Reference
    - 8.1 [Authentication](#81-authentication)
    - 8.2 [Projects](#82-projects)
    - 8.3 [Proposals](#83-proposals)
    - 8.4 [CV Management](#84-cv-management)
    - 8.5 [Messaging](#85-messaging)
    - 8.6 [Users](#86-users)
9. Data Models
10. Security
11. Future Improvements

---

1. Project Overview

FreelanceHub is a full-stack web platform that connects freelancers with clients who need to hire talent for short- and long-term projects. The backend, built with Python/Flask, exposes a RESTful JSON API consumed by a React-based frontend.

The platform enables clients to publish projects, receive and manage proposals from freelancers, exchange messages, and track milestones. Freelancers can browse open projects, submit proposals, maintain a portfolio of CVs, and communicate directly with potential clients.

---

2. Objectives

#	Objective
1	Provide a secure, token-based authentication system for two user roles: **freelancer** (`work`) and **client** (`hire`).
2	Allow clients to create, update, and delete projects with budgets, deadlines, skills, and milestones.
3	Allow freelancers to browse available projects and submit competitive proposals.
4	Offer a real-time–ready messaging system for direct communication between users.
5	Let freelancers upload and manage their CV documents (stored as base64 in MongoDB).
6	Deliver paginated, filterable, and searchable API responses suitable for a modern SPA.

---

3. Features and Functionalities

3.1 User Management

- Registration with role selection (`work` = freelancer, `hire` = client)
- Login with JWT access token response
- Fetch authenticated user profile (`/api/auth/me`)
- Update profile fields: name, role, bio, avatar, password

3.2 Project Management

- Create projects (clients only) with title, budget, deadline, skills, milestones, and attachments
- List all projects with pagination, status filter, category filter, and skill filter
- Full-text search across title, client name, category, and skills
- View single project by ID
- Update project fields (owner only)
- Update project status: `pending` → `in-progress` → `completed` (owner only)
- Delete project and its associated proposals (owner only)

3.3 Proposal System

- Submit proposals on open projects (freelancers only)
- Include cover letter, bid amount, and estimated delivery time

- List proposals for a project (project owner only)
- Update or withdraw a proposal
- Accept / reject a proposal (status transitions)
- Proposal count on a project is automatically kept in sync

3.4 CV Management

- Upload a CV as base64-encoded data
- List all CVs belonging to the authenticated user
- Mark a CV as primary
- Download a CV (returns base64 payload)
- Delete a CV

3.5 Messaging

- Send a message to another user
- Retrieve the full conversation thread between two users
- List all conversations for the authenticated user (with last message preview)

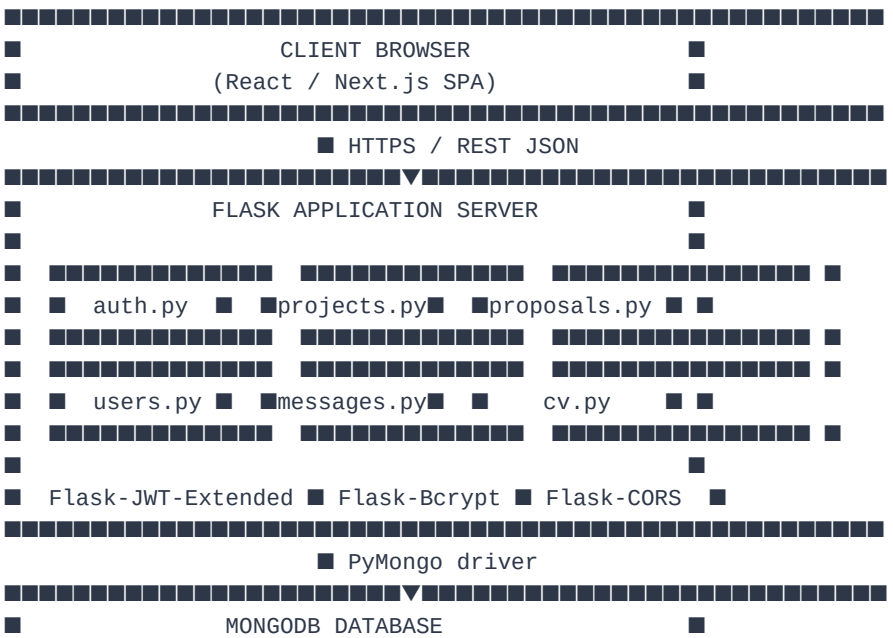
3.6 User Discovery

- List all freelancers publicly (no auth required)
- List users available for messaging (auth required, filterable by role)

---

4. System Architecture

4.1 High-Level Overview



```
■  Collections: users ■ projects ■ proposals      ■
■               cvs   ■ messages                  ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

4.2 Application Module Structure

```
BackendFreelance/
■■■ run.py                  # Entry point – creates and runs the Flask app
■■■ requirements.txt        # Python dependencies
■■■ .env                    # Environment variables (not committed)
■■■ app/
■   ■■■ __init__.py         # Application factory, extension init, blueprint registration
■   ■■■ app.py              # Duplicate entry point (dev convenience)
■   ■■■ config.py           # Configuration class (Mongo URI, JWT secret)
■   ■■■ models/
■   ■   ■■■ user.py         # UserModel  – CRUD + serialization
■   ■   ■■■ project.py      # ProjectModel + ProposalModel
■   ■   ■■■ message.py      # MessageModel
■   ■   ■■■ proposals.py    # (legacy / extended proposal helpers)
■   ■■■ routes/
■       ■■■ auth.py         # /api/auth/*
■       ■■■ projects.py     # /api/projects/*
■       ■■■ proposals.py    # /api/proposals/*
■       ■■■ cv.py           # /api/cv/*
■       ■■■ messages.py     # /api/messages/* and /api/conversations
■       ■■■ users.py        # /api/users/*
■■■ docs/
    ■■■ Project_Documentation.md  ← this file
    ■■■ Project_Documentation.pdf
```

4.3 Request Life-Cycle

```
HTTP Request
    ■
    ▼
Flask Router  ■■■ Blueprint Route Handler
                      ■
                      ■■■ JWT Validation (flask-jwt-extended)
                      ■
                      ■■■ Input Validation / Business Rules
                      ■
                      ■■■ Model Layer  (PyMongo calls)
                      ■
                      ■■■ JSON Response
```

---

5. Technologies Used

5.1 Backend

Library	Version	Purpose
Python	3.10+	Runtime language
Flask	3.1.3	Micro web framework
Flask-JWT-Extended	4.7.1	JWT authentication
Flask-Bcrypt	1.0.1	Password hashing
Flask-PyMongo	3.0.1	MongoDB ODM integration
Flask-CORS	6.0.2	Cross-Origin Resource Sharing
PyMongo	4.16.0	Low-level MongoDB driver
PyJWT	2.11.0	JWT encoding/decoding
python-dotenv	1.2.2	Environment variable loading

5.2 Database

Technology	Details
MongoDB	NoSQL document store (local or Atlas)
Database name	`freelancehub`
Collections	`users`, `projects`, `proposals`, `cvs`, `messages`

5.3 Frontend (expected consumer)

Technology	Purpose
React / Next.js	SPA consuming this API
Axios / Fetch	HTTP client
JWT stored in localStorage / httpOnly cookie	Auth token management

5.4 Development & Tooling

Tool	Purpose
Git + GitHub	Version control
python-dotenv	Per-environment secrets
Postman / Thunder Client	API testing

6. Installation and Setup

6.1 Prerequisites

- Python 3.10 or higher
- MongoDB 6.x running locally (or a MongoDB Atlas connection string)
- `pip` (Python package manager)

6.2 Clone the Repository

```
git clone https://github.com/yahyaouiiii/BackendFreelance.git
cd BackendFreelance
```

6.3 Create and Activate a Virtual Environment

```
python3 -m venv venv
# Linux / macOS
source venv/bin/activate
# Windows
venv\Scripts\activate
```

6.4 Install Dependencies

```
pip install -r requirements.txt
```

6.5 Configure Environment Variables

Create a `.env` file in the project root:

```
MONGO_URI=mongodb://localhost:27017/freelancehub
JWT_SECRET_KEY=replace_with_a_strong_random_secret
```

***Note:** Never commit `.env` to version control. The `.gitignore` already excludes it.*

6.6 Start MongoDB

```
# Linux / macOS (homebrew)
brew services start mongodb-community
# or
mongod --dbpath /data/db
```

6.7 Run the Application

```
python run.py
```

The API server will be available at `http://127.0.0.1:5000`.

---

7. Usage Guide

7.1 Registering a User

Send a POST request to `/api/auth/register` with a JSON body:

```
{
  "name": "Alice Dupont",
  "email": "alice@example.com",
  "password": "SecurePass123!",
  "role": "work"
}
```

`role` accepts `"work"` (freelancer) or `"hire"` (client).

The response includes a JWT access token:

```
{
  "token": "<jwt>",
  "user": { "id": "...", "name": "Alice Dupont", "email": "...", "role": "work" }
}
```

7.2 Authenticating Subsequent Requests

Include the token in the `Authorization` header:

```
Authorization: Bearer <jwt>
```

7.3 Creating a Project (client)

```
POST /api/projects
Authorization: Bearer <token>

{
  "title": "Build a portfolio website",
  "budget": 500,
  "deadline": "2026-07-01",
  "category": "Web Development",
  "skills": ["React", "CSS", "Node.js"],
  "description": ["Modern single-page portfolio", "Responsive design required"]
}
```

7.4 Submitting a Proposal (freelancer)

```
POST /api/proposals
Authorization: Bearer <token>

{
  "projectId": "<project_id>",
  "bidAmount": 450,
  "deliveryTime": "3 weeks",
```

```
  "coverLetter": "I have 5 years of React experience..."
}
```

---

8. API Endpoints Reference

***Base URL:** `http://localhost:5000`*

***Auth:** Endpoints marked ■ require `Authorization: Bearer `.*

8.1 Authentication

Method	Endpoint	Auth	Description
POST	`/api/auth/register`	—	Register a new user
POST	`/api/auth/login`	—	Login and receive JWT
GET	`/api/auth/me`	■	Get authenticated user profile
PUT	`/api/auth/update`	■	Update profile (name, bio, avatar, password)

POST /api/auth/register — Request Body

```
{ "name": "string", "email": "string", "password": "string", "role": "work|hire" }
```

POST /api/auth/login — Request Body

```
{ "email": "string", "password": "string" }
```

8.2 Projects

Method	Endpoint	Auth	Description
GET	`/api/projects`	—	List all projects (paginated)
GET	`/api/projects/my`	■	Client: own projects; Freelancer: available projects
GET	`/api/projects/search?q=`	—	Full-text search
GET	`/api/projects/`	—	Get single project
POST	`/api/projects`	■ (hire)	Create a project

Method	Endpoint	Auth	Description
PUT	`/api/projects/`	■ (owner)	Update project fields
PATCH	`/api/projects//status`	■ (owner)	Change project status
DELETE	`/api/projects/`	■ (owner)	Delete project

Query Parameters for GET /api/projects

Param	Type	Default	Description
page	int	1	Page number
limit	int	10	Items per page
status	string	—	Filter by `pending`, `in-progress`, `completed`
category	string	—	Filter by category (regex)
skill	string	—	Filter by skill (regex)

8.3 Proposals

Method	Endpoint	Auth	Description
GET	`/api/proposals/`	■ (project owner)	List proposals for a project
GET	`/api/proposals/detail/`	■	Get single proposal
POST	`/api/proposals`	■ (work)	Submit a proposal
PUT	`/api/proposals/`	■	Update proposal (coverLetter, bidAmount, deliveryTime)
PATCH	`/api/proposals//status`	■	Change status: `pending`, `accepted`, `rejected`, `withdrawn`
DELETE	`/api/proposals/`	■	Delete a proposal

8.4 CV Management

Method	Endpoint	Auth	Description
POST	`/api/cv/upload`	■	Upload a CV (base64 encoded)
GET	`/api/cv`	■	List all CVs for current user
PUT	`/api/cv//primary`	■	Set a CV as primary
GET	`/api/cv//download`	■	Download a CV (base64 response)
DELETE	`/api/cv/`	■	Delete a CV

POST /api/cv/upload — Request Body

```
{ "name": "my_cv.pdf", "data": "<base64_string>", "size": 204800 }
```

8.5 Messaging

Method	Endpoint	Auth	Description
GET	`/api/conversations`	■	List all conversations (with last message)
GET	`/api/messages/`	■	Get full thread with a user
POST	`/api/messages`	■	Send a message

POST /api/messages — Request Body

```
{ "recipientId": "<user_id>", "text": "Hello, I'm interested in your project." }
```

8.6 Users

Method	Endpoint	Auth	Description
GET	`/api/users/freelancers`	—	List all public freelancer profiles
GET	`/api/users/available`	■	List users for messaging (optional `role` filter)

---

9. Data Models

9.1 User

```
_id          ObjectId
name         string
email        string  (unique)
password     string  (bcrypt hash)
role         string  ("work" | "hire")
subscription string  ("free" | "premium")
bio          string
avatar       string  (URL or base64)
created_at   datetime
```

9.2 Project

```
_id             ObjectId
title           string
client          string        (display name)
client_id       string        (User._id)
client_avatar   string
status          string        ("pending" | "in-progress" | "completed")
budget          float
deadline        string
progress        int           (0–100)
category        string
duration        string
proposals_count int
description     [string]
skills          [string]
attachments     [object]
milestones      [{id, title, status, payment}]
created_at      datetime
updated_at      datetime
```

9.3 Proposal

```
_id            ObjectId
project_id     string
freelancer_id  string
cover_letter   string
bid_amount     float
delivery_time  string
status         string  ("pending" | "accepted" | "rejected" | "withdrawn")
created_at     datetime
updated_at     datetime
```

9.4 CV

```
_id         ObjectId
user_id     string
name        string
data        string  (base64)
size        int     (bytes)
isPrimary   bool
uploadDate  string
```

9.5 Message

```
_id           ObjectId
sender_id     string
recipient_id  string
text          string
created_at    datetime
```

---

10. Security

Concern	Mitigation
Password storage	bcrypt hashing via Flask-Bcrypt
Authentication	Stateless JWT (Flask-JWT-Extended); tokens expire based on config
Authorization	Role checks and ownership checks in every protected route
CORS	Controlled via Flask-CORS extension
Secret management	Secrets loaded from `.env`, never committed to source control
Input validation	Required-field checks and type coercions before every DB write

> ***Production note:** Before deploying, set `JWT_SECRET_KEY` to a cryptographically random value, configure `JWT_ACCESS_TOKEN_EXPIRES`, and use HTTPS.*

---

11. Future Improvements

Priority	Feature
High	WebSocket-based real-time messaging (Socket.IO)
High	Email notifications on proposal acceptance / new message
High	Premium subscription management with Stripe integration
Medium	Admin dashboard and moderation tools
Medium	Rating and review system for completed projects
Medium	Advanced search with Elasticsearch / Atlas Search

Priority	Feature
Low	OAuth 2.0 social login (Google, GitHub)
Low	Multi-language (i18n) support
Low	Mobile applications (React Native)

Documentation generated for the FreelanceHub project — May 2026.